scRNA Seurat Analysis

2026-03-26

Important commands in scRNA Seurat analysis

```
Read10X()
CreateSeuratObject() # for each replicate
PercentageFeatureSet() # to add % mitochondrial gene
subset() # filter outliner for each replicate
list() # combine control and disease data into a list
NormalizeData() # inside lapply(), for each dataset independently
FindVariableFeatures() # inside lapply(), for each dataset independently
SelectIntegrationFeatures() # select features that are repeatedly variable across replicates
FindIntegrationAnchors() # correct technical difference among replicates
IntegrateData()
DefaultAssay() <- 'integrated'

ScaleData()
RunPCA() # stored in CONIPF.combined@reduction$pca
ElbowPlot() # to see how many PCs to include in the next steps
FindNeighbors()
FindClusters()
RunUMAP() # stored in CONIPF.combined@reductions$umap
RunTSNE() # stored in CONIPF.combined@reductions$tsne, Safe to run sequentially after RunUMAP

DimPlot() # UMAP plot

DotPlot() # show gene expression across clusters
RenameIdents() # rename clusters
FeaturePlot() # show gene's location in what cluster
VlnPlot()
DoHeatmap()
```

Load Libraries

```
# Install packages from CRAN
install.packages(c("Seurat", "tidyverse", "patchwork"))
```

```
library(Seurat)
library(tidyverse)
library(patchwork)
```

Read barcodes, features (genes), matrix files

Follow this video (https://youtu.be/43Z13DS_emQ?si=23ASY9qQ-Rk5ek5-) and this video (https://youtu.be/6ANenznQZ94?si=Dtj8f90r7PWeGiD_) to download GSE132771 data from GEO Accession Viewer, then format them correctly before reading them into R.

There're 3 Control (NML) replicates and 3 Disease (IPF) replicates to be downloaded.

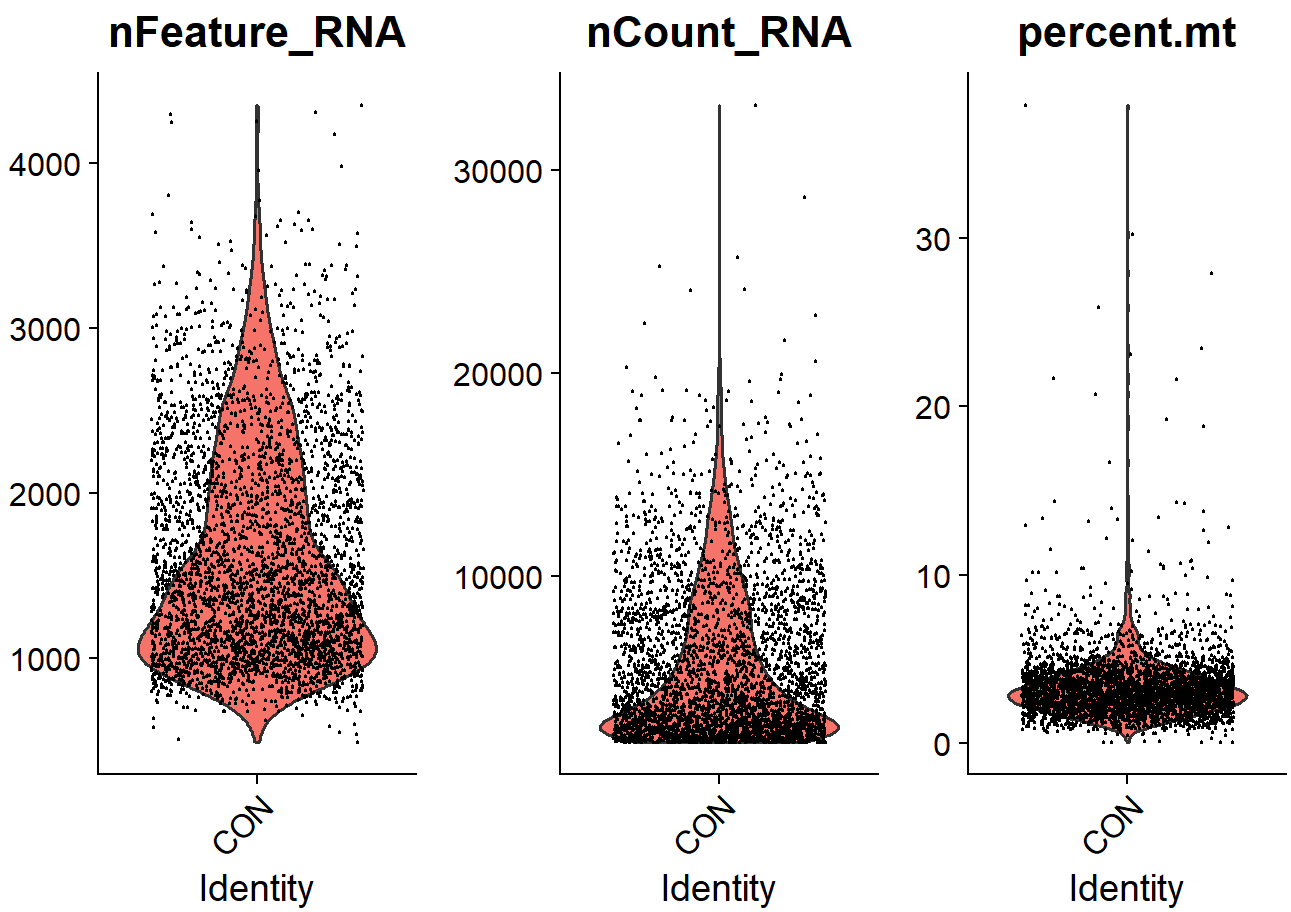
```
CON_1 <- Read10X(data.dir = "GSE132771_RAW/NML1")
CON_2 <- Read10X(data.dir = "GSE132771_RAW/NML2")
CON_3 <- Read10X(data.dir = "GSE132771_RAW/NML3")

IPF_1 <- Read10X(data.dir = "GSE132771_RAW/IPF1")
IPF_2 <- Read10X(data.dir = "GSE132771_RAW/IPF2")
IPF_3 <- Read10X(data.dir = "GSE132771_RAW/IPF3")
```

Create Seurat object and add percent mitochondrial genes

```
CON_1 <- CreateSeuratObject(counts = CON_1, project = "CON", min.cells = 3, min.features = 200)
CON_1 <- PercentageFeatureSet(CON_1, pattern = "^MT-", col.name = "percent.mt")

CON_2 <- CreateSeuratObject(counts = CON_2, project = "CON", min.cells = 3, min.features = 200)
CON_2 <- PercentageFeatureSet(CON_2, pattern = "^MT-", col.name = "percent.mt")

CON_3 <- CreateSeuratObject(counts = CON_3, project = "CON", min.cells = 3, min.features = 200)
CON_3 <- PercentageFeatureSet(CON_3, pattern = "^MT-", col.name = "percent.mt")


IPF_1 <- CreateSeuratObject(counts = IPF_1, project = "IPF", min.cells = 3, min.features = 200)
IPF_1 <- PercentageFeatureSet(IPF_1, pattern = "^MT-", col.name = "percent.mt")

IPF_2 <- CreateSeuratObject(counts = IPF_2, project = "IPF", min.cells = 3, min.features = 200)
IPF_2 <- PercentageFeatureSet(IPF_2, pattern = "^MT-", col.name = "percent.mt")

IPF_3 <- CreateSeuratObject(counts = IPF_3, project = "IPF", min.cells = 3, min.features = 200)
IPF_3 <- PercentageFeatureSet(IPF_3, pattern = "^MT-", col.name = "percent.mt")
```

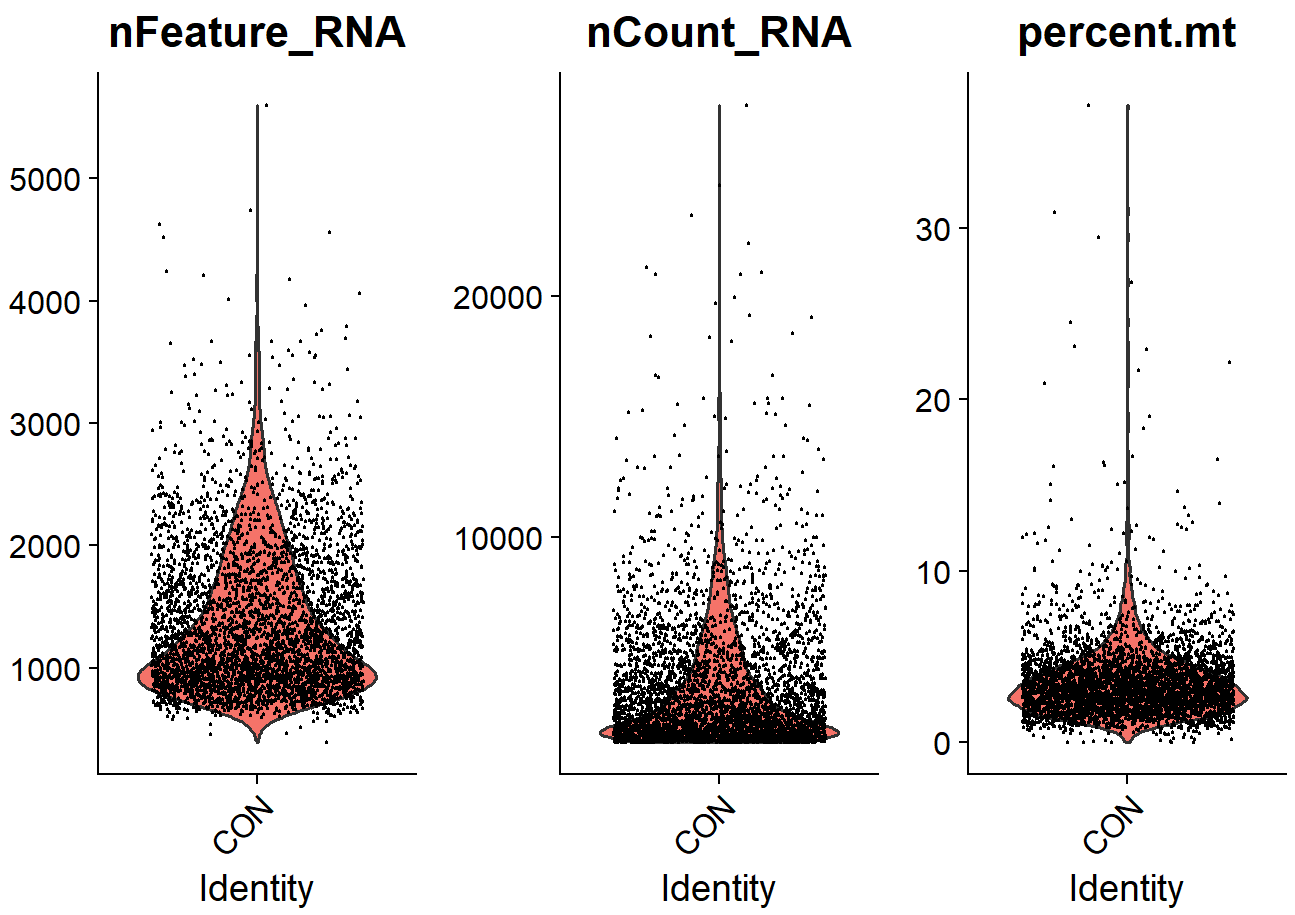
Visualize data distribution to filter low quality cells

```
VlnPlot(CON_1, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
```



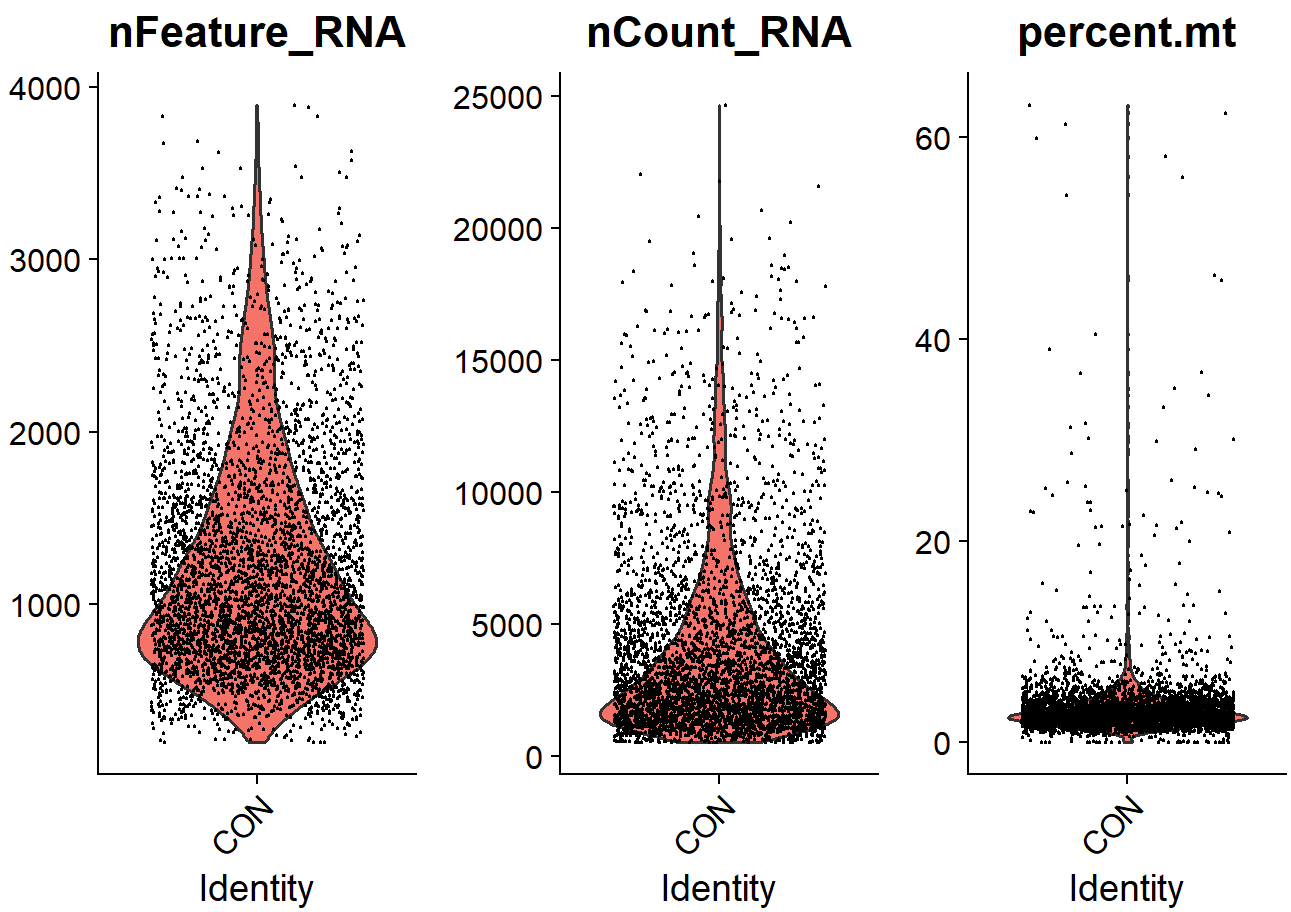

```
CON_1 <- subset(CON_1, subset = nFeature_RNA < 4000 & nCount_RNA < 20000 & percent.mt < 10)
```

```
VlnPlot(CON_2, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
```



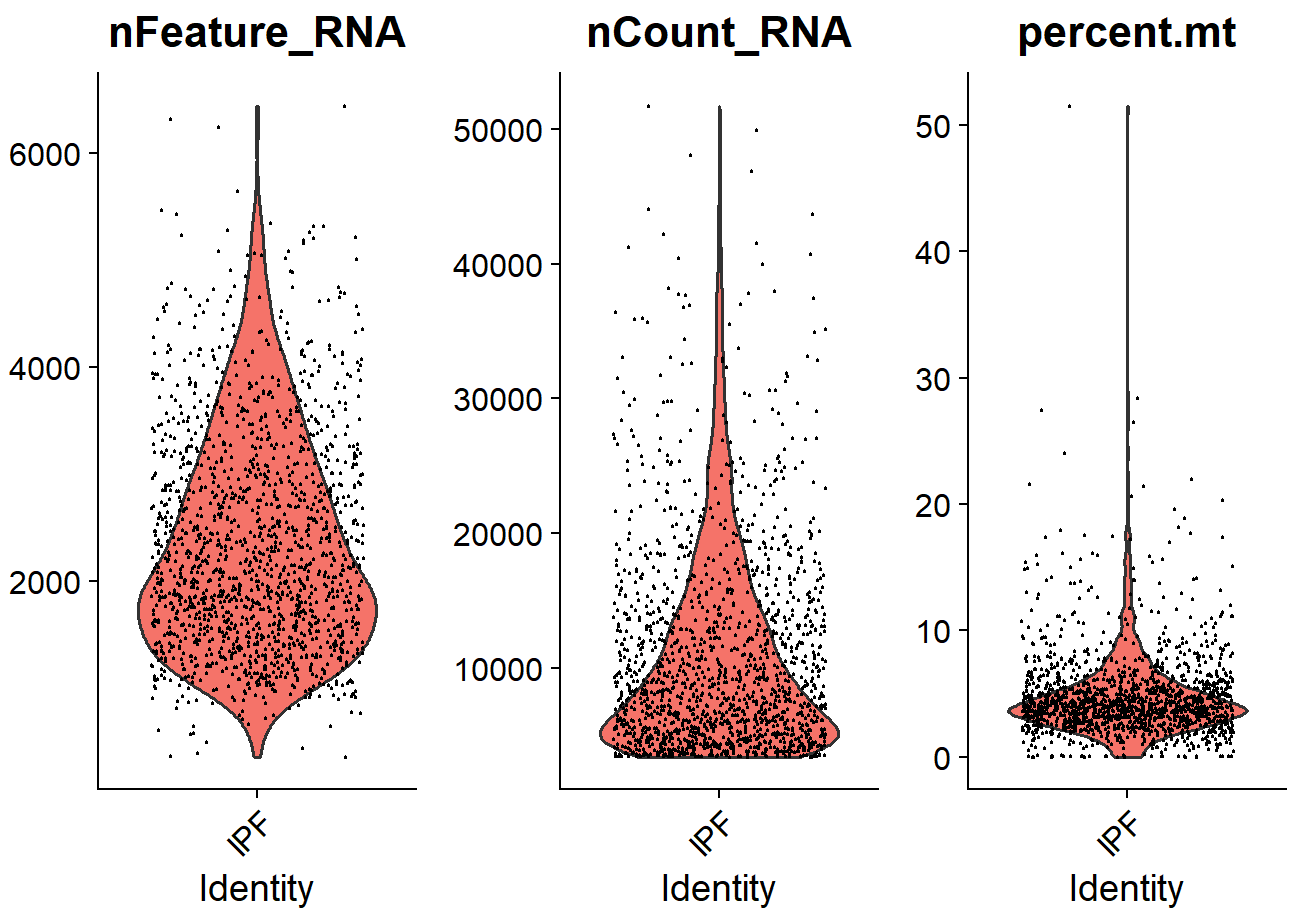

```
CON_2 <- subset(CON_2, subset = nFeature_RNA < 4000 & nCount_RNA < 10000 & percent.mt < 10)
```

```
VlnPlot(CON_3, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
```



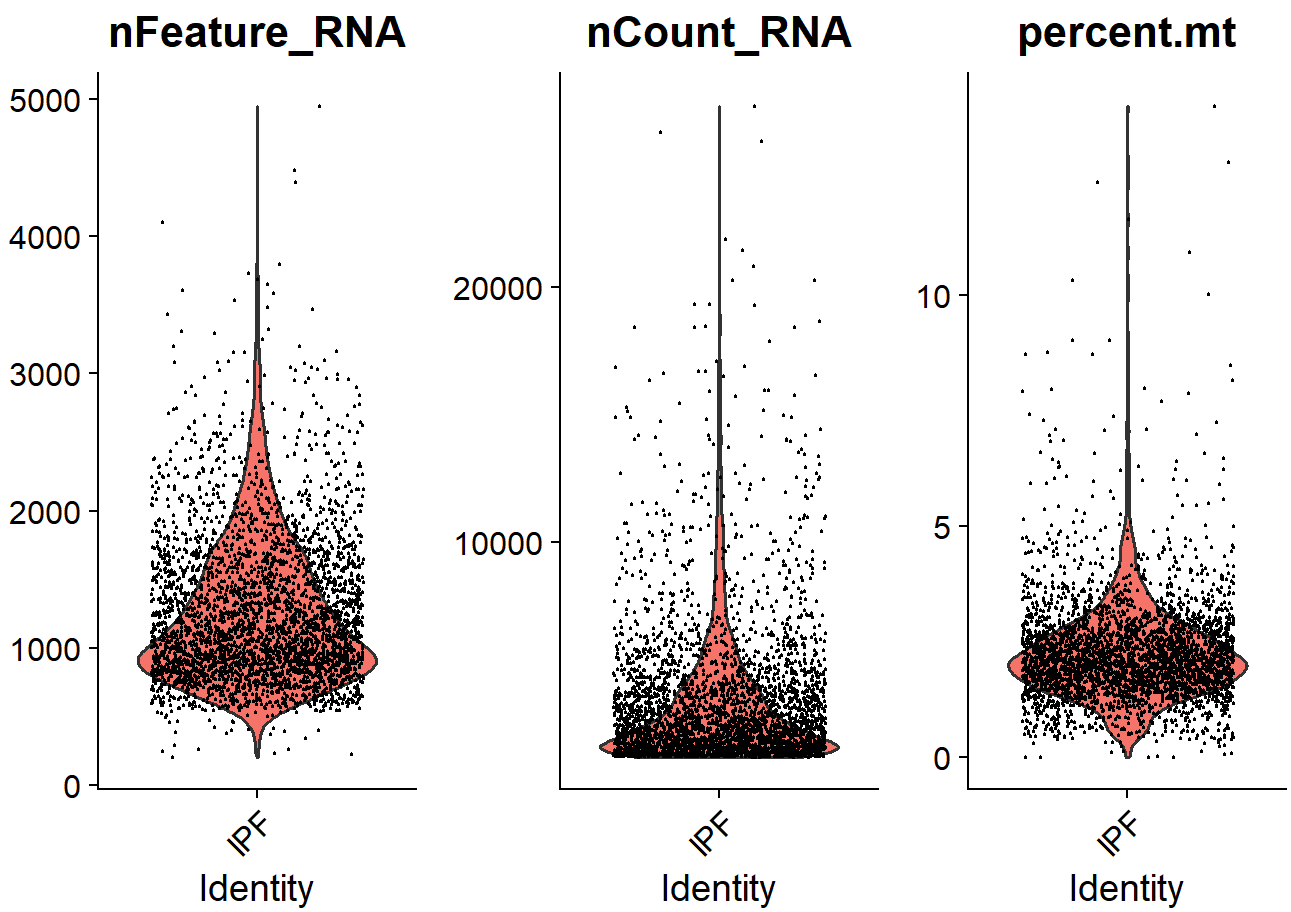

```
CON_3 <- subset(CON_3, subset = nFeature_RNA < 4000 & nCount_RNA < 20000 & percent.mt < 10)
```

```
VlnPlot(IPF_1, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
```



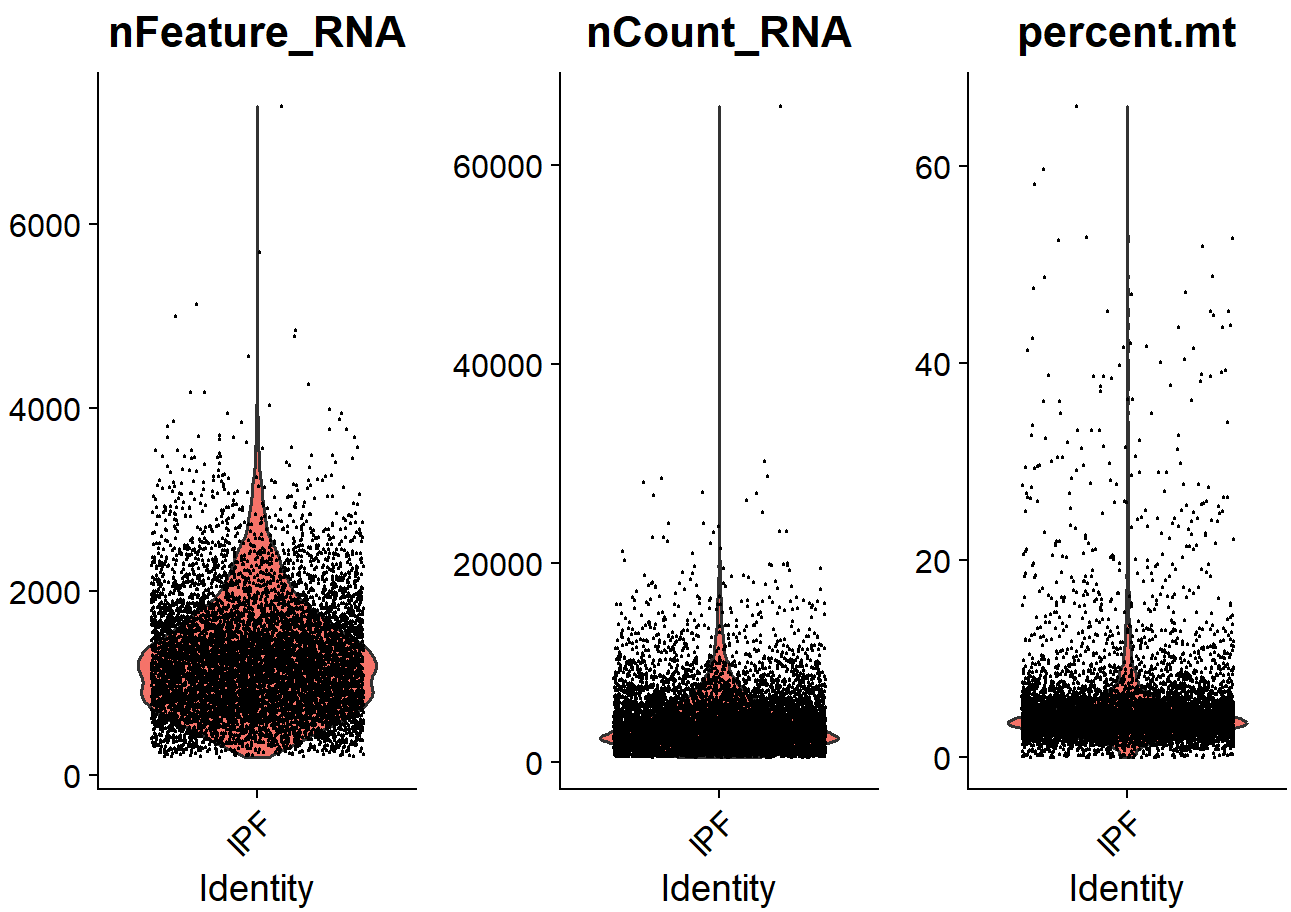

```
IPF_1 <- subset(IPF_1, subset = nFeature_RNA < 6000 & nCount_RNA < 30000 & percent.mt < 10)
```

```
VlnPlot(IPF_2, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
```




```
IPF_2 <- subset(IPF_2, subset = nFeature_RNA < 3000 & nCount_RNA < 15000 & percent.mt < 5)
```

```
VlnPlot(IPF_3, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
```




```
IPF_3 <- subset(IPF_3, subset = nFeature_RNA < 4000 & nCount_RNA < 20000 & percent.mt < 10)
```

Combine control and disease data into a list

```
CONIPF.list <- list(CON_1 = CON_1, CON_2 = CON_2, CON_3 = CON_3,
                    IPF_1 = IPF_1, IPF_2 = IPF_2, IPF_3 = IPF_3)
```

Normalize and identify variable features for each dataset

```
CONIPF.list <- lapply(X = CONIPF.list, FUN = function(x) {
  x <- NormalizeData(x)
  x <- FindVariableFeatures(x, selection.method = "vst", nfeatures = 2000)
})
```

Select features that are repeatedly variable across datasets for integration

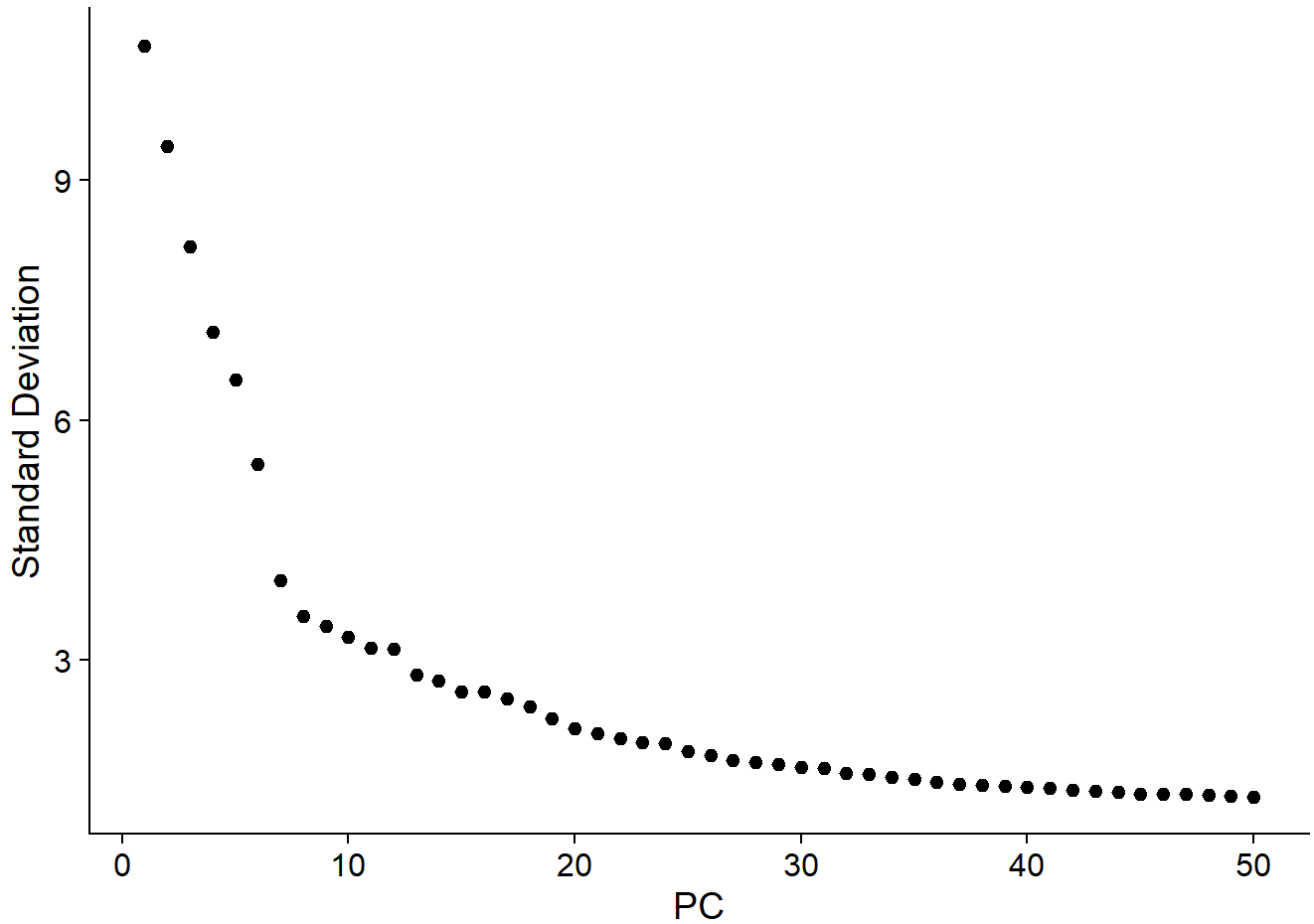
```
features <- SelectIntegrationFeatures(object.list = CONIPF.list)
CONIPF.anchors <- FindIntegrationAnchors(object.list = CONIPF.list, anchor.features = features)
```

Creates an ‘integrated’ data assay

```
CONIPF.combined <- IntegrateData(anchorset = CONIPF.anchors)

DefaultAssay(CONIPF.combined) <- "integrated"
```

Run the standard workflow for visualization and clustering

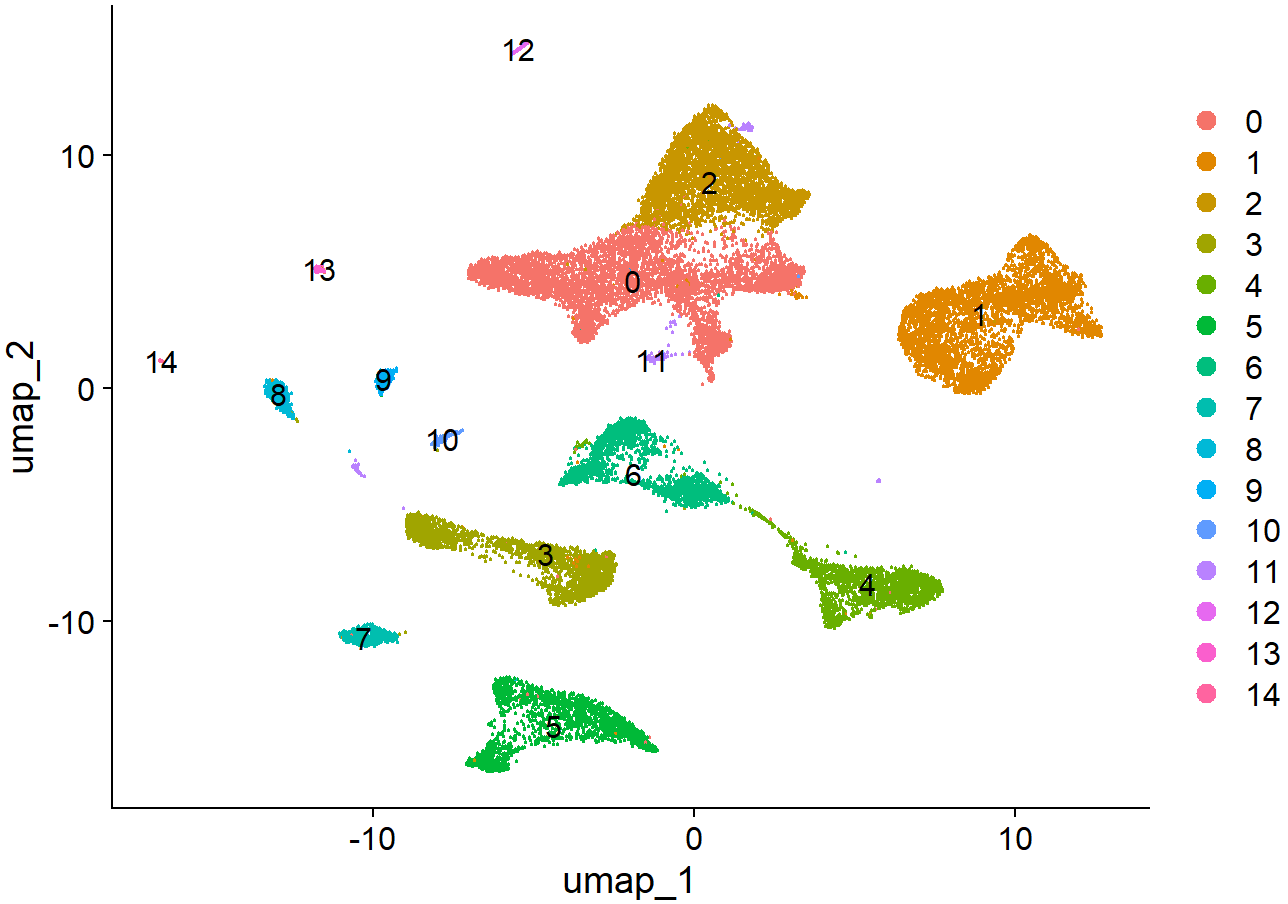
```
CONIPF.combined <- ScaleData(CONIPF.combined, verbose = FALSE)
CONIPF.combined <- RunPCA(CONIPF.combined, npcs = 50, verbose = FALSE)
ElbowPlot(CONIPF.combined, ndims = 50, reduction = 'pca')
```



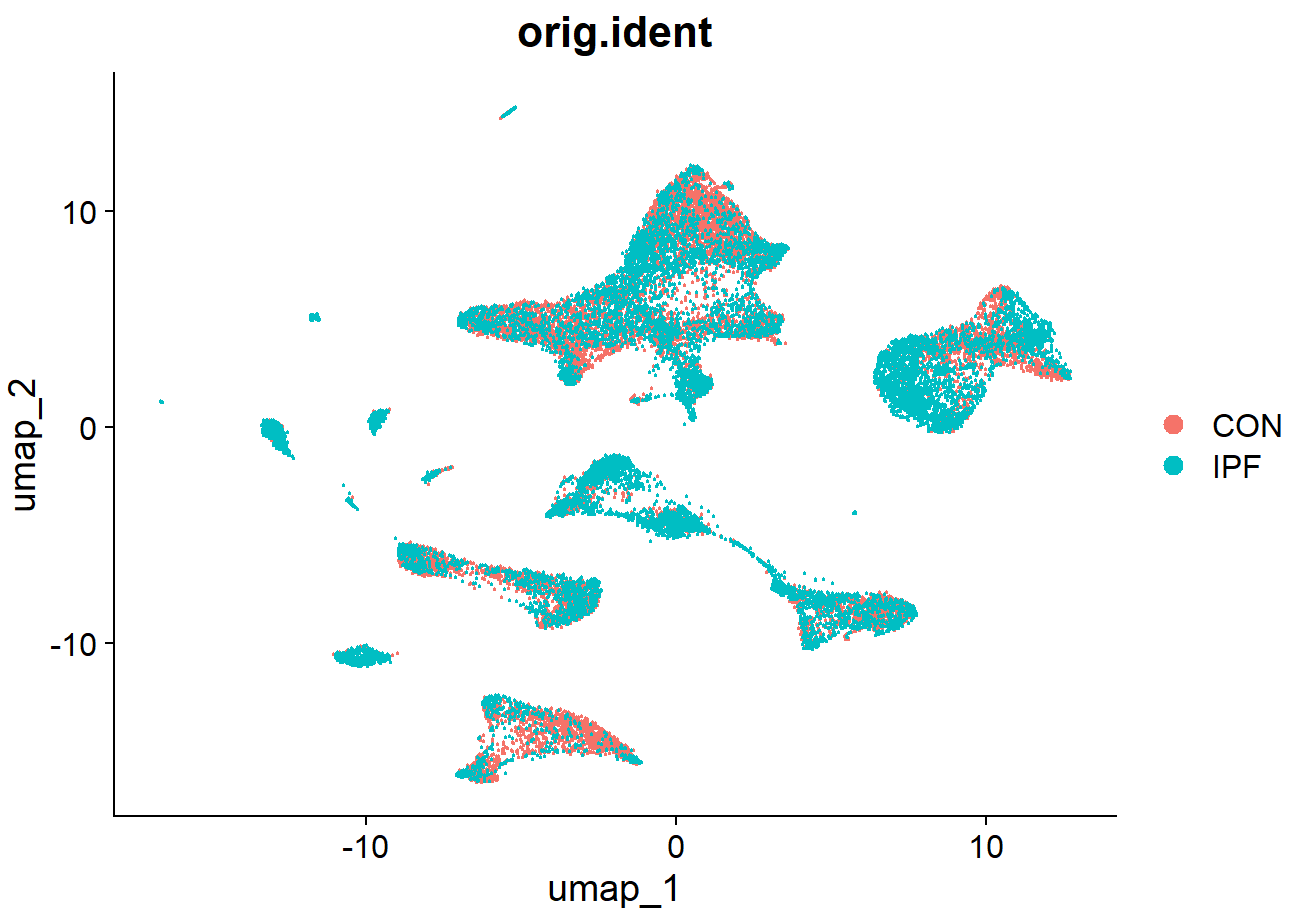

```
CONIPF.combined <- RunUMAP(CONIPF.combined, reduction = "pca", dims = 1:30)
CONIPF.combined <- FindNeighbors(CONIPF.combined, reduction = "pca", dims = 1:30)
CONIPF.combined <- FindClusters(CONIPF.combined, resolution = 0.1)
```

```
## Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
##
## Number of nodes: 24314
## Number of edges: 1049396
##
## Running Louvain algorithm...
## Maximum modularity in 10 random starts: 0.9803
## Number of communities: 15
## Elapsed time: 4 seconds
```

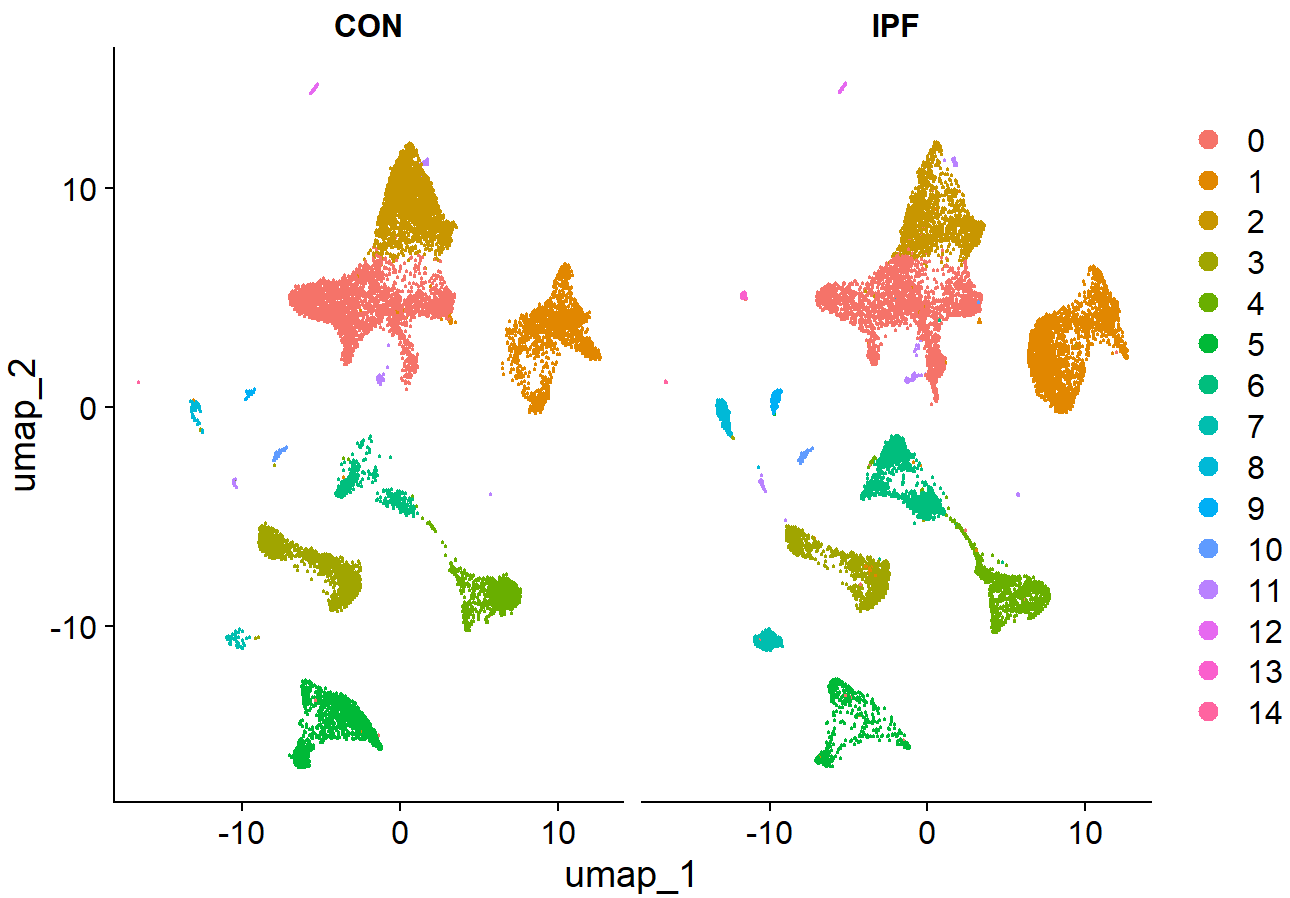
```
DimPlot(CONIPF.combined, reduction = "umap", label = TRUE) # 15 clusters
```




```
DimPlot(CONIPF.combined, reduction = "umap", group.by = "orig.ident")
```

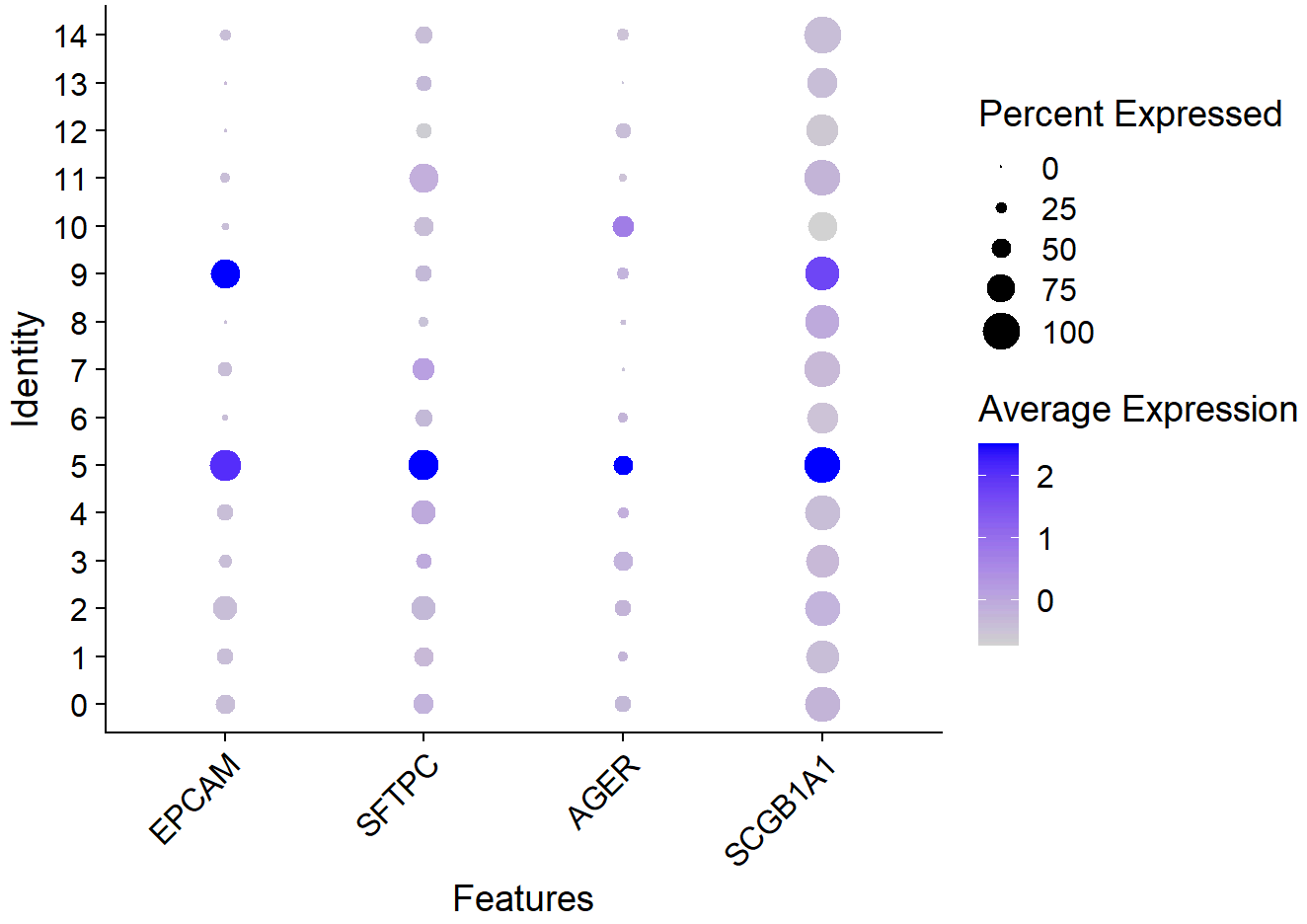



```
DimPlot(CONIPF.combined, reduction = "umap", split.by = "orig.ident")
```



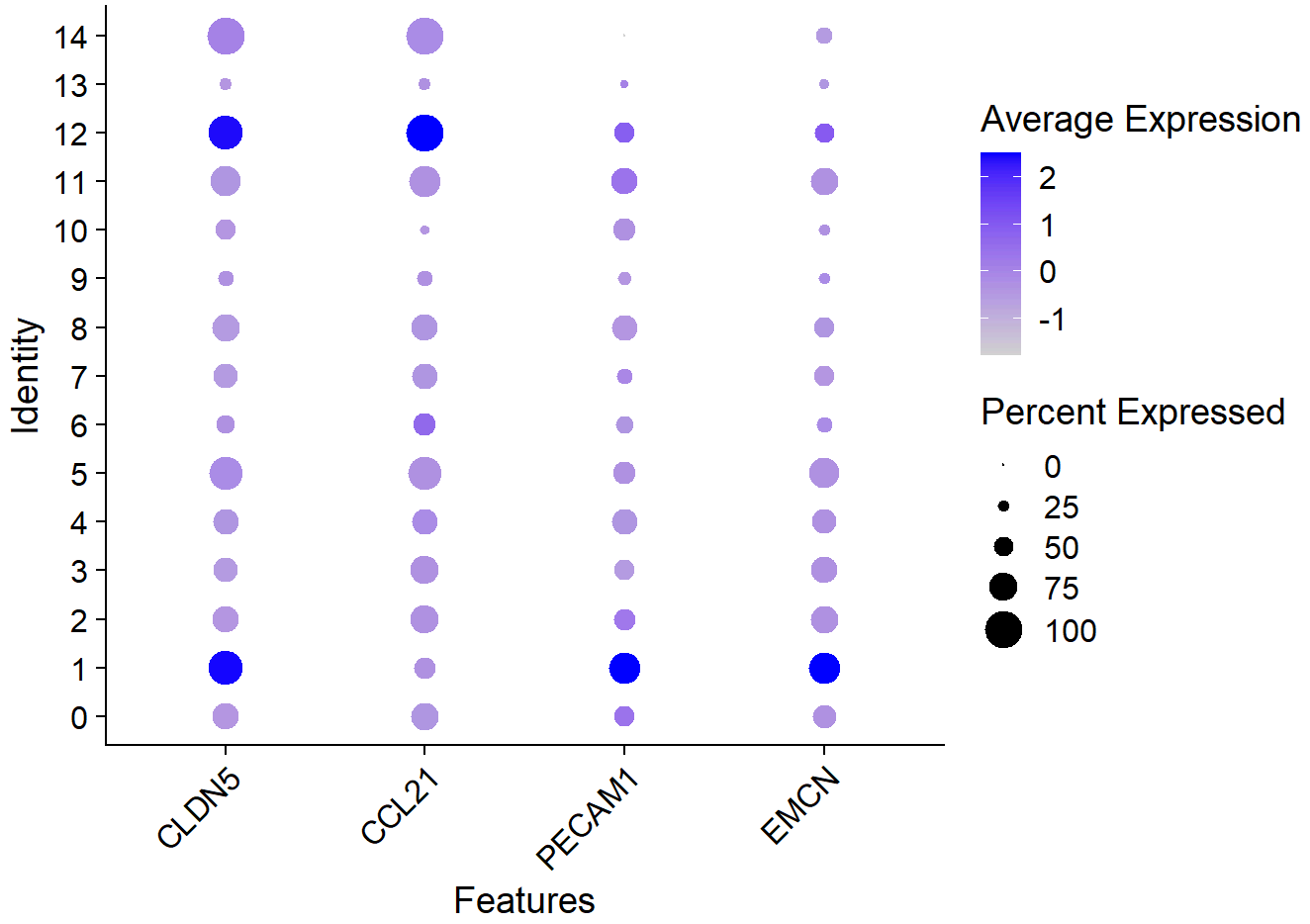
Identify which cluster has the enriched amount of certain marker genes

```
DotPlot(CONIPF.combined, features = c("EPCAM", "SFTPC", "AGER", "SCGB1A1")) + RotatedAxis()
```



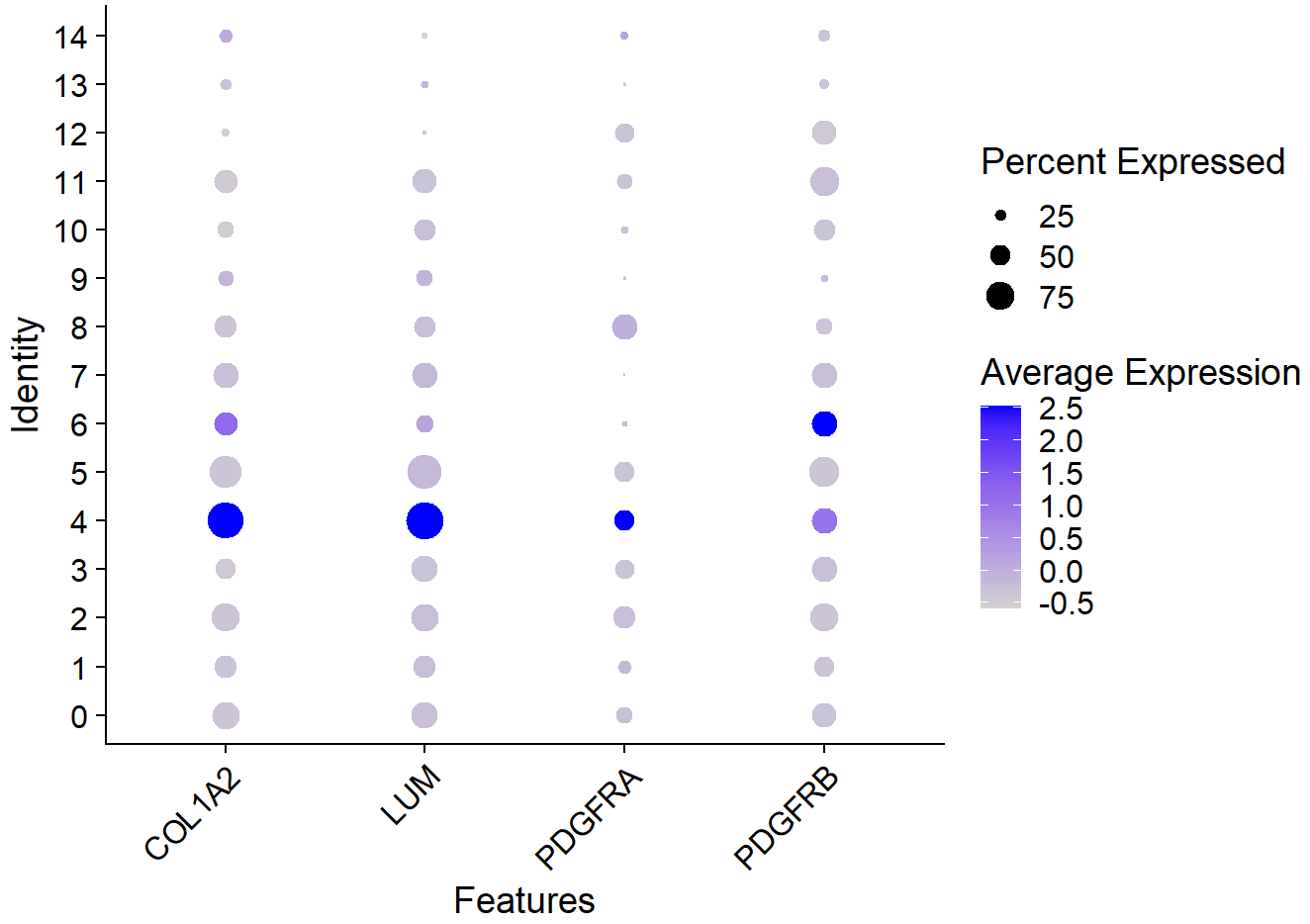
“EPCAM”, “SFTPC”, “AGER”, “SCGB1A1” –> Epithelial cell –> clusters 5, 9

```
DotPlot(CONIPF.combined, features = c("CLDN5", "CCL21", "PECAM1", "EMCN")) + RotatedAxis()
```



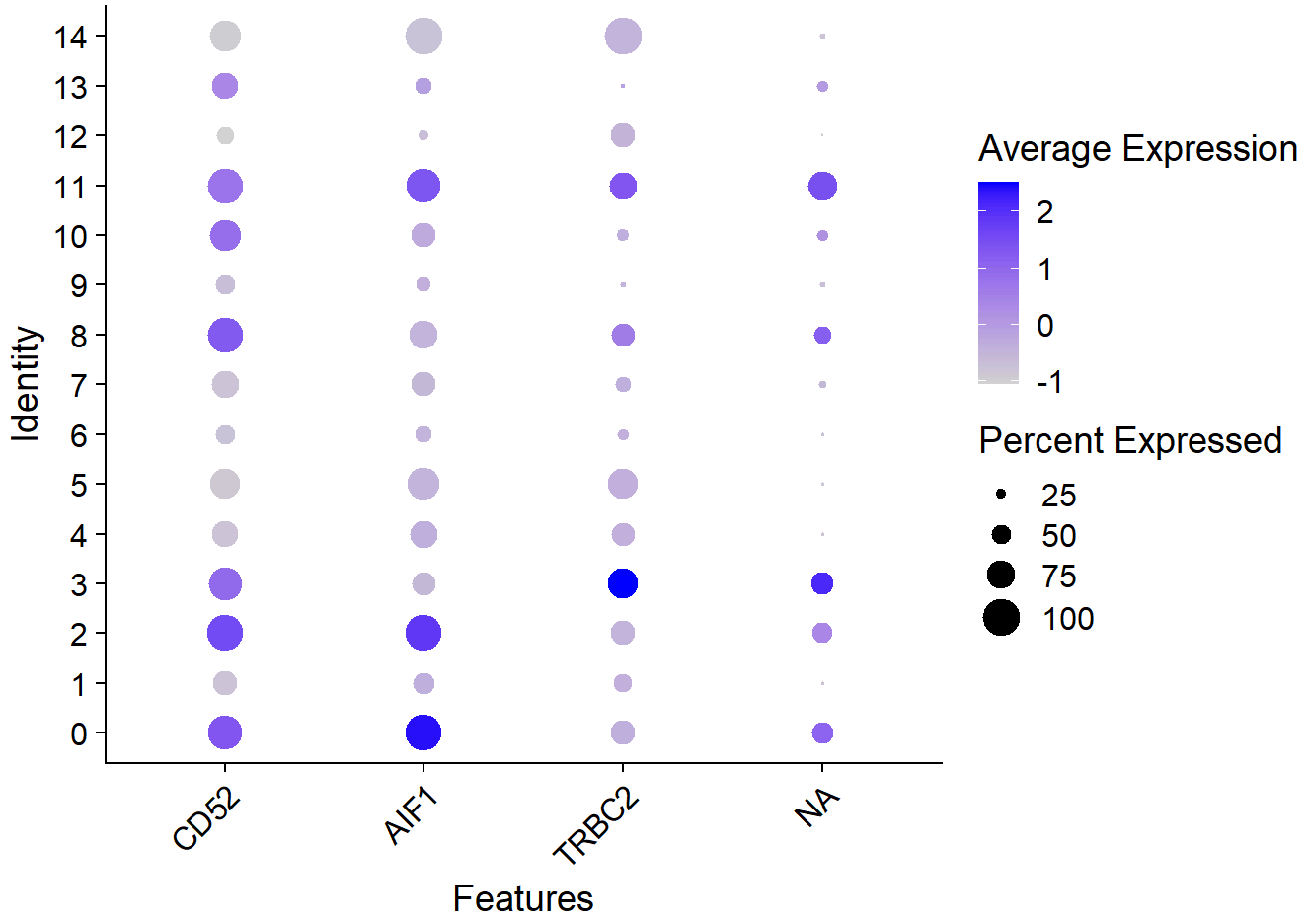
“CLDN5”, “CCL21”, “PECAM1”, “EMCN” –> Endothelial cell –> clusters 1, 12

```
DotPlot(CONIPF.combined, features = c("COL1A2", "LUM", "PDGFRA","PDGFRB")) + RotatedAxis()
```



“COL1A2”, “LUM”, “PDGFRA”,“PDGFRB” –> Mesenchymal cell –> clusters 4, 6, 14

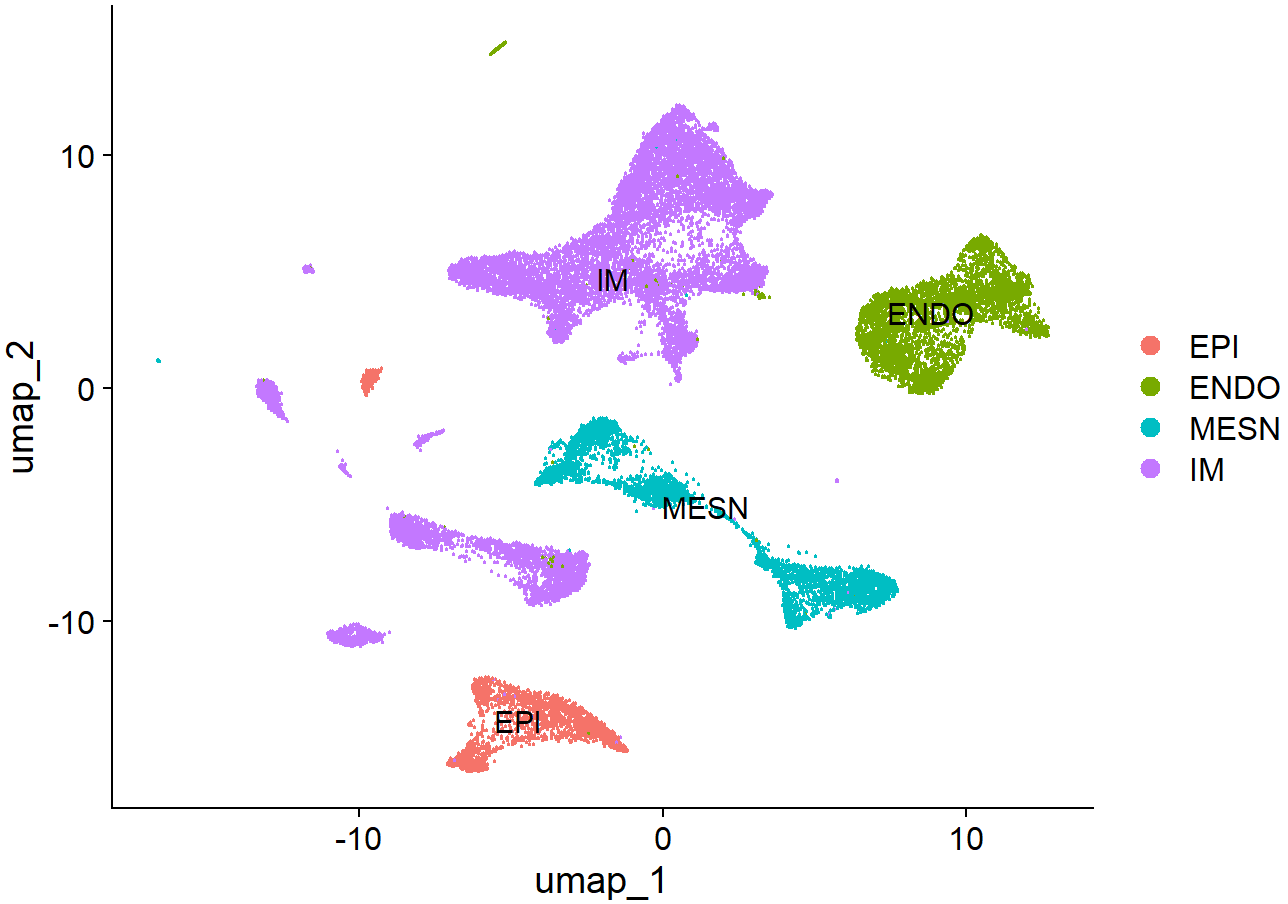
```
DotPlot(CONIPF.combined, features = c("PTPRC", "CD52", "AIF1", "TRBC2")) + RotatedAxis()
```



“PTPRC”, “CD52”, “AIF1”, “TRBC2” –> Immune cell –> clusters 0, 2, 3, 7, 8, 10, 11, 13

Rename cluster number with cell type name

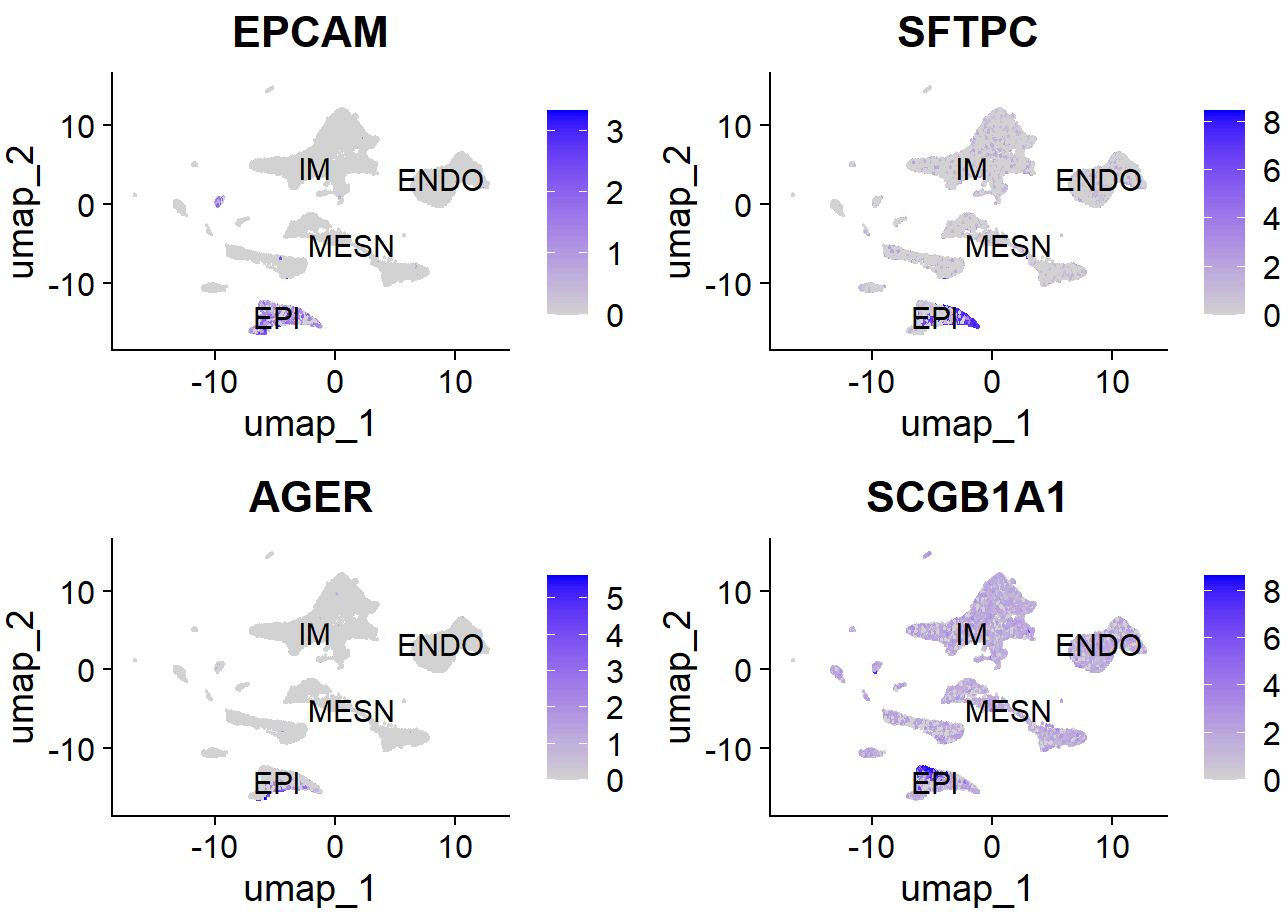
```
CONIPF.combined <- RenameIdents(CONIPF.combined,
                                `5` = "EPI", `9` = "EPI", `1` = "ENDO",`12` = "ENDO",
                                `4` = "MESN", `6` = "MESN", `14` = "MESN",
                                `0` = "IM", `2` = "IM", `3` = "IM", `7` = "IM",
                                `8` = "IM",`10` = "IM",`11` = "IM",`13` = "IM")

DimPlot(CONIPF.combined, reduction = "umap", label = TRUE)
```



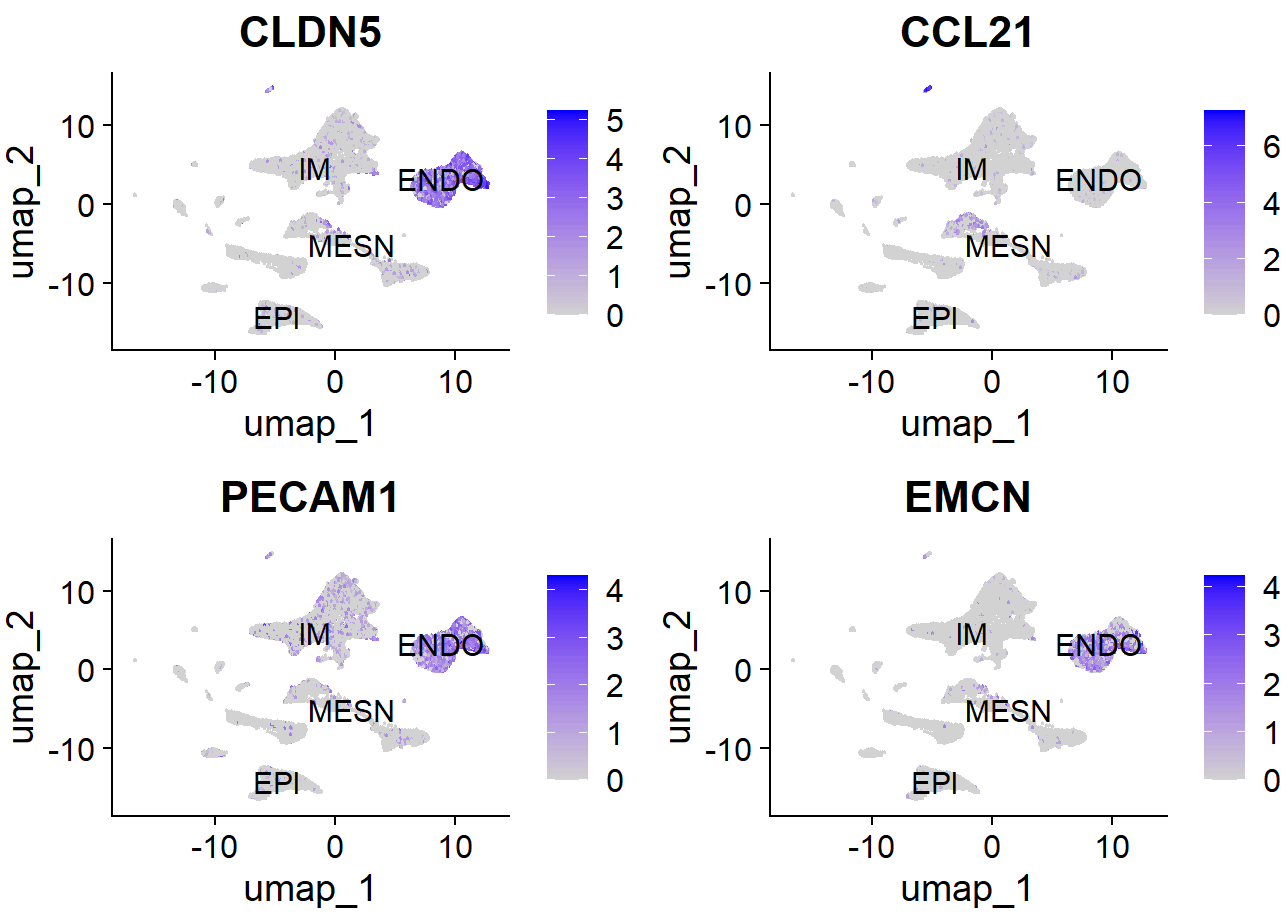
FeaturePlot()

Visualize the location of these marker genes in UMAP plot

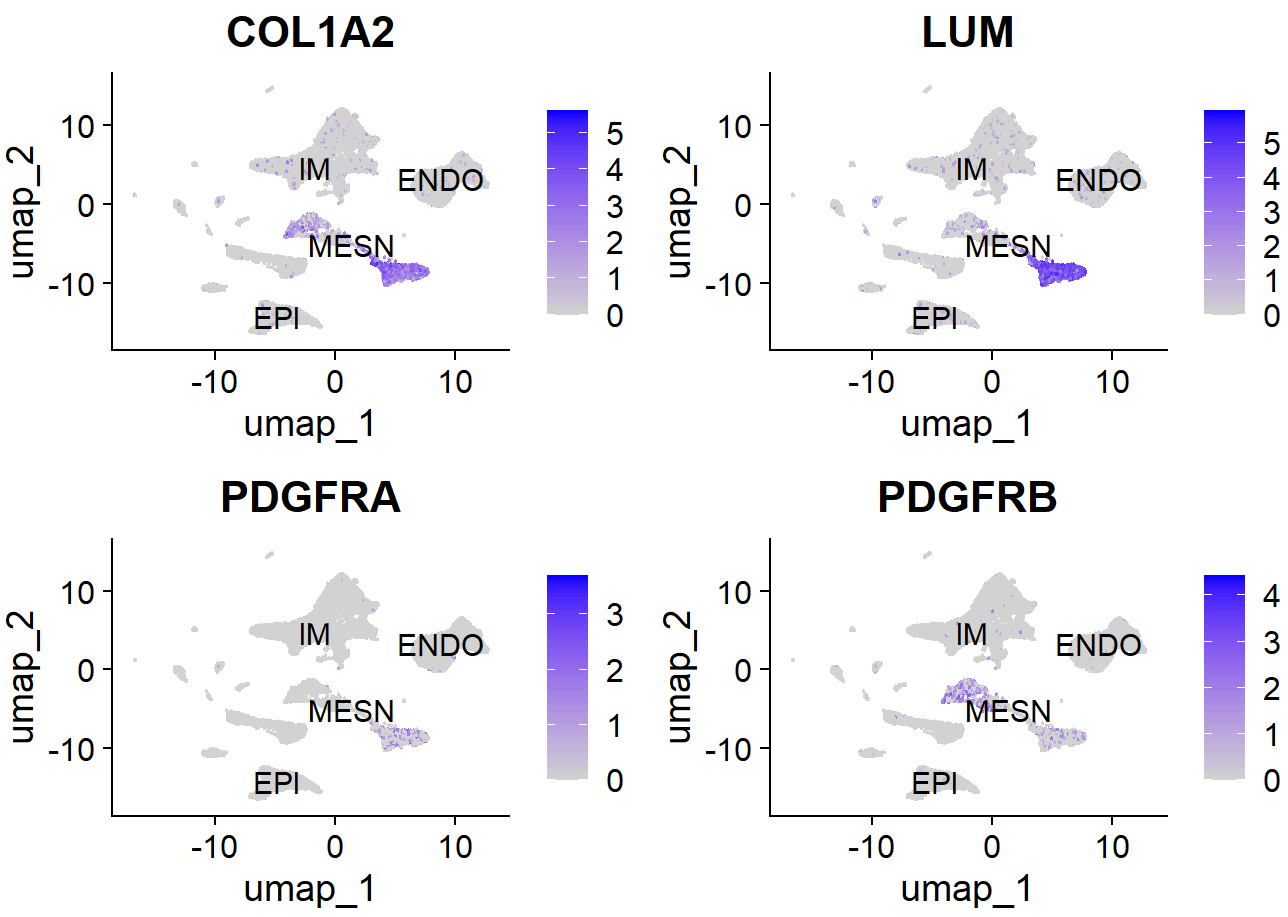
```
DefaultAssay(CONIPF.combined) <- "RNA"

FeaturePlot(CONIPF.combined, features=c("EPCAM", "SFTPC", "AGER", "SCGB1A1"),
            label=TRUE, cols = c('lightgrey', 'blue'))
```



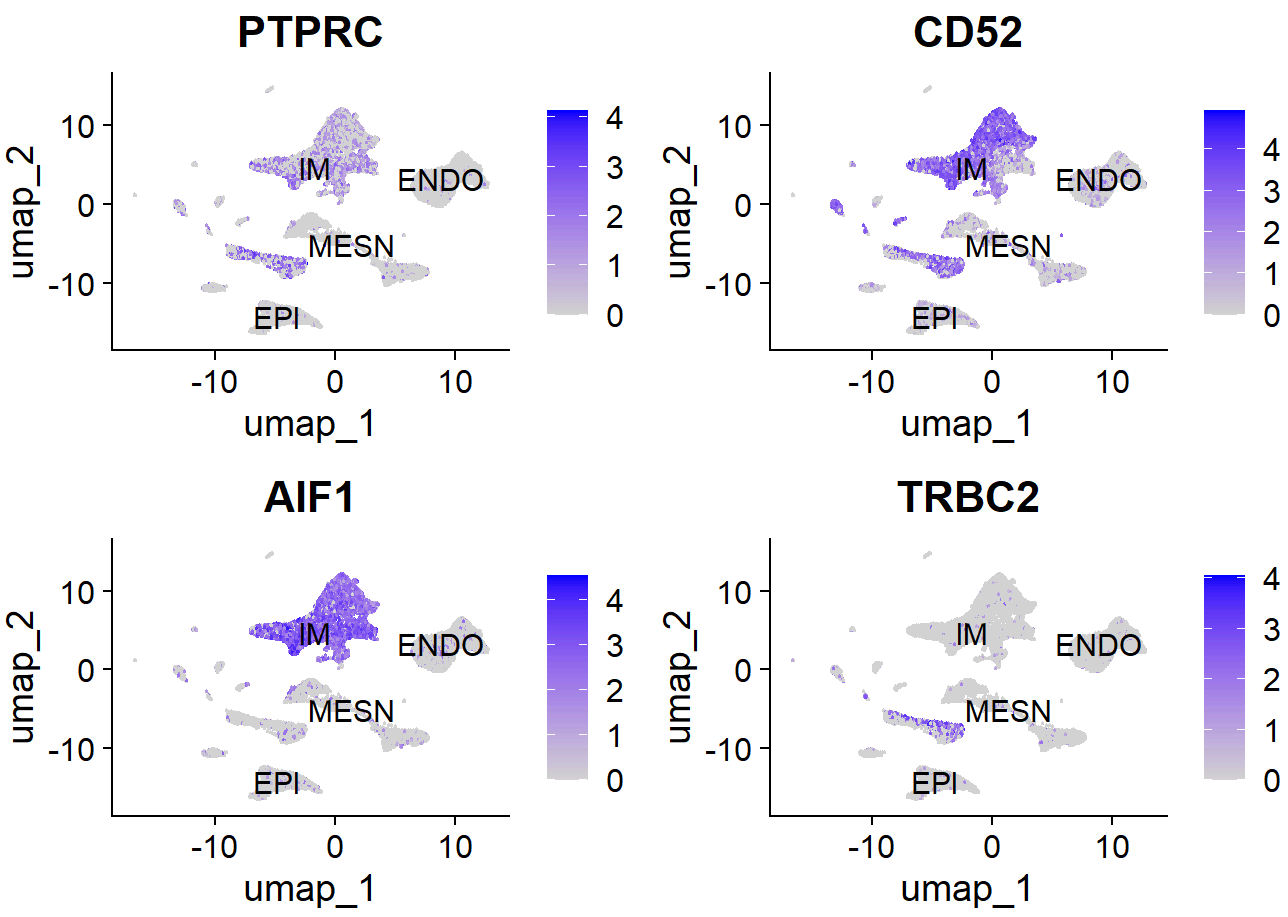

```
FeaturePlot(CONIPF.combined, features=c("CLDN5", "CCL21", "PECAM1", "EMCN"),
            label=TRUE, cols = c('lightgrey', 'blue'))
```




```
FeaturePlot(CONIPF.combined, features=c("COL1A2", "LUM", "PDGFRA","PDGFRB"),
            label=TRUE, cols = c('lightgrey', 'blue'))
```

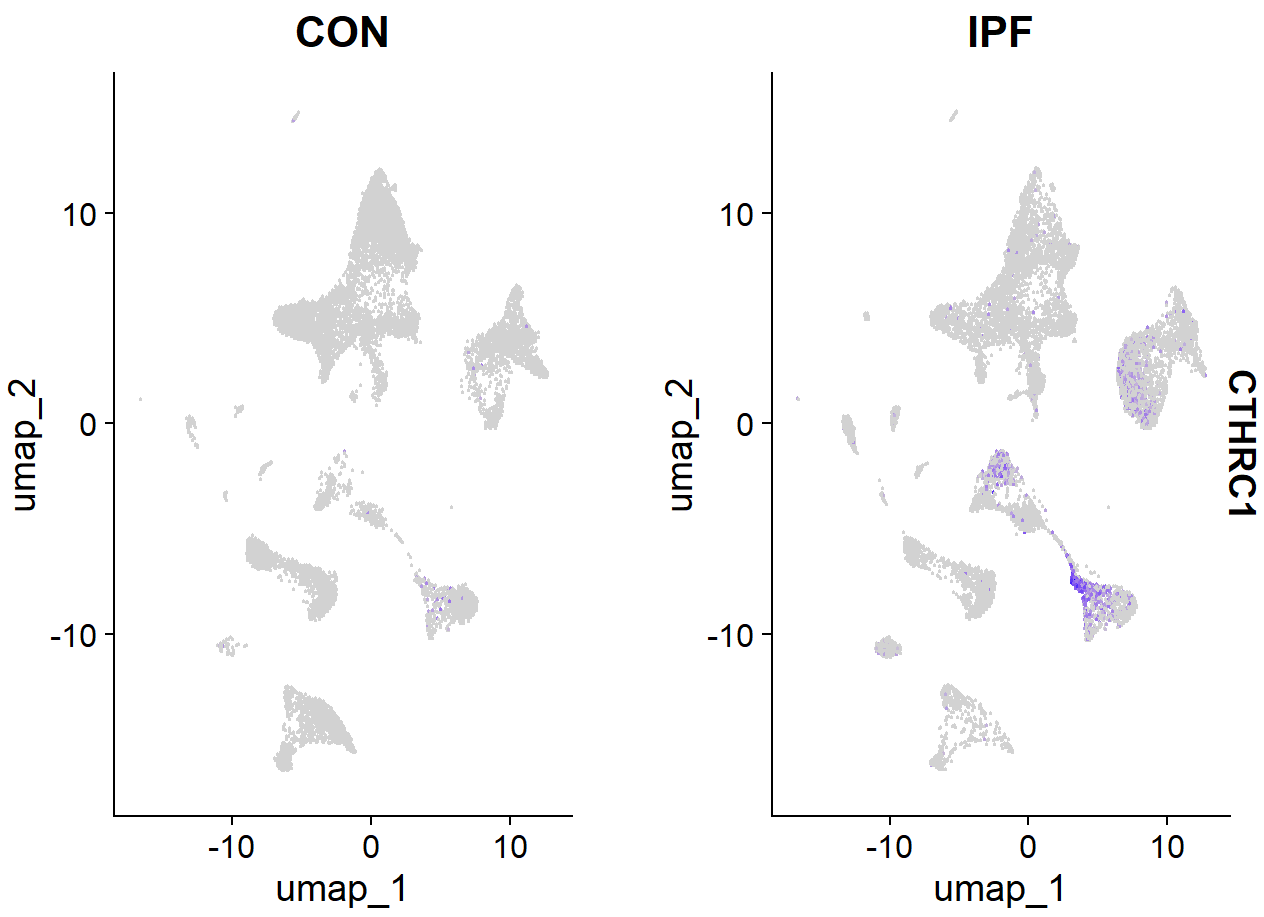



```
FeaturePlot(CONIPF.combined, features=c("PTPRC", "CD52", "AIF1", "TRBC2"),
            label=TRUE, cols = c('lightgrey', 'blue'))
```



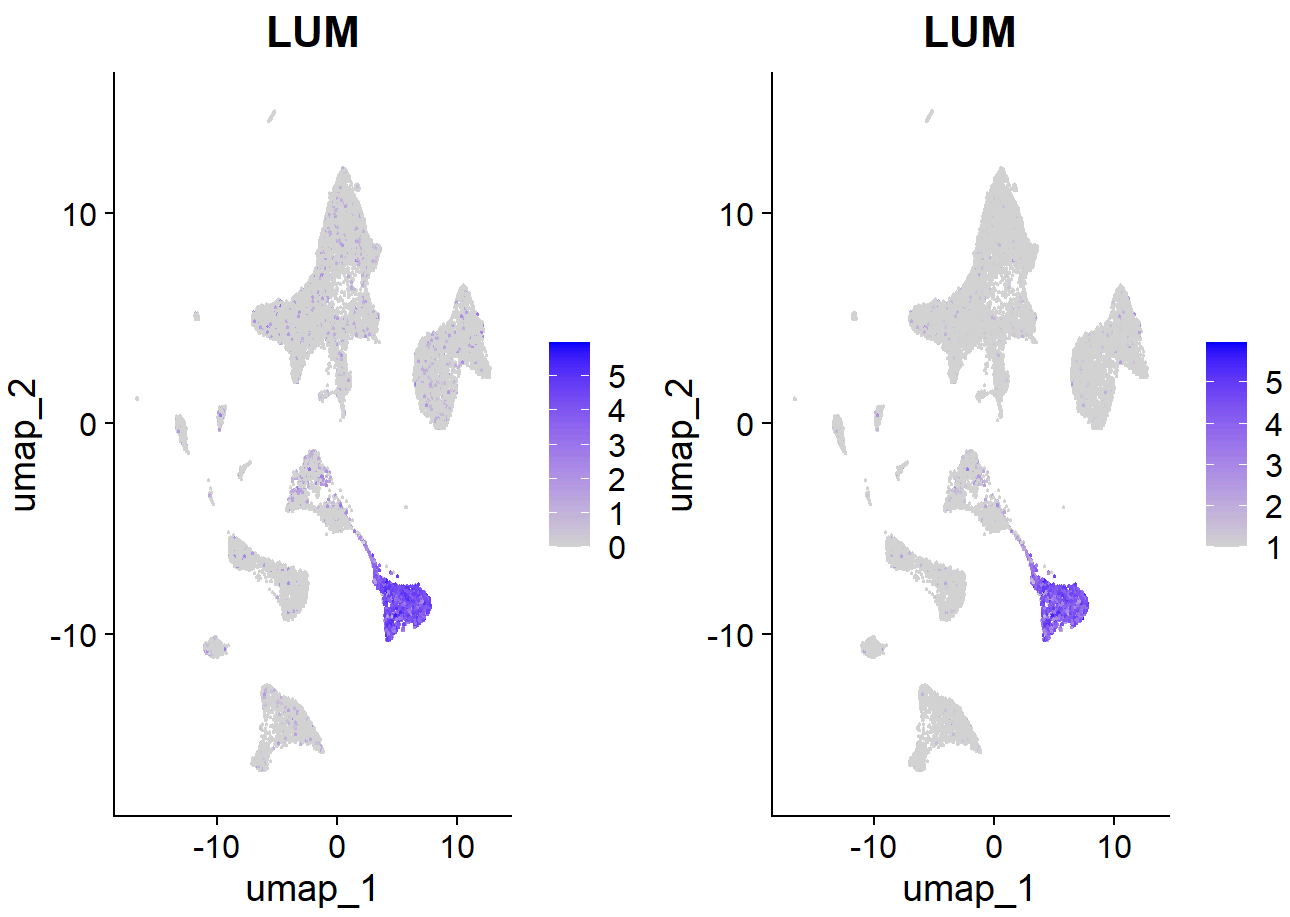
Compare CTHRC1 gene expression in control vs disease

```
FeaturePlot(CONIPF.combined, features = "CTHRC1", split.by = "orig.ident")
```



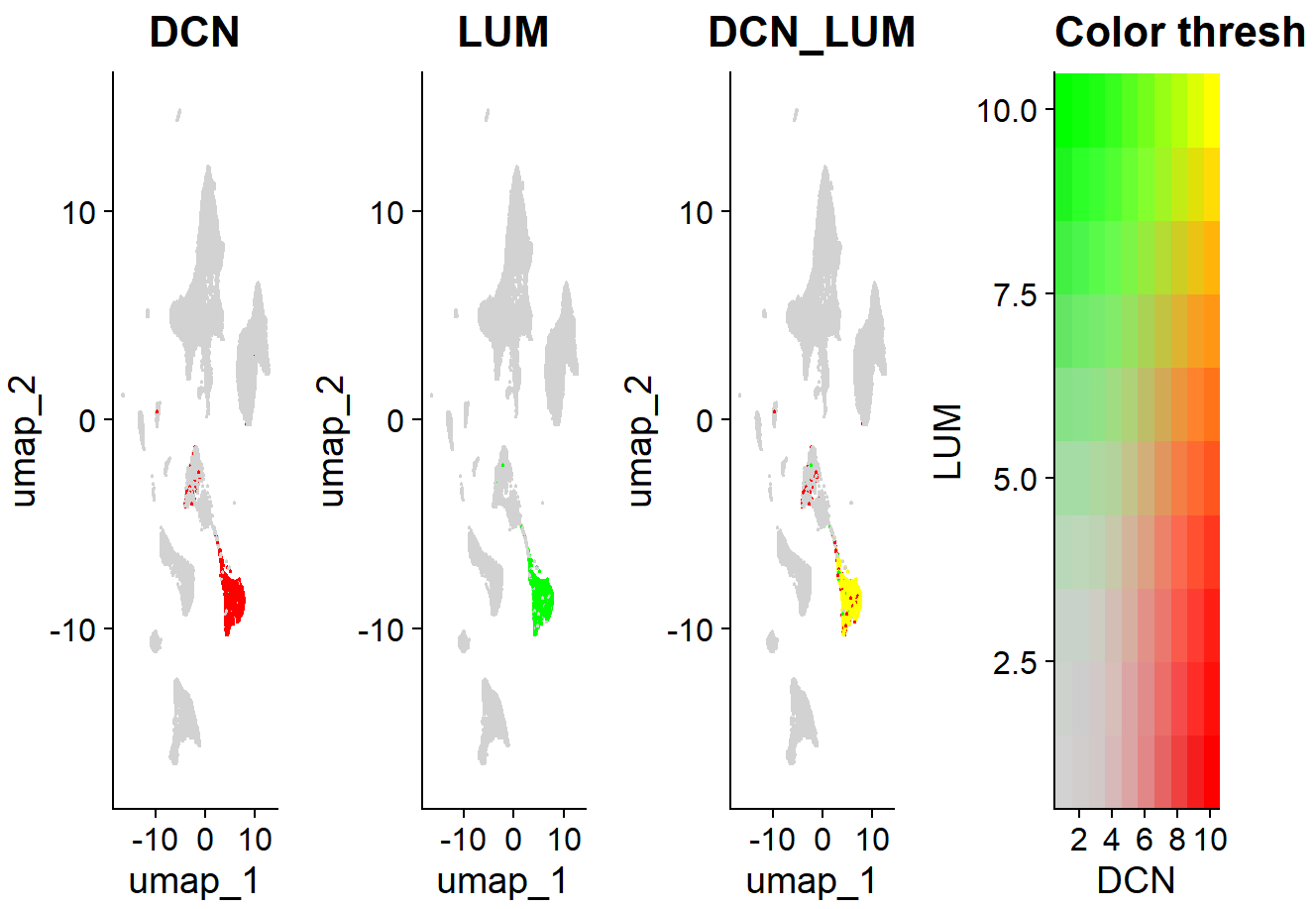
Adjust the contrast in the plot

```
P1 <- FeaturePlot(CONIPF.combined, features = "LUM") # expression level 0-5 is shown
P2 <- FeaturePlot(CONIPF.combined, features ="LUM", min.cutoff =1) # so that only expression lev
el 1-5 is shown
P1|P2 # plots 1 and 2 side by side
```



Visualize co-expression of two features (genes)

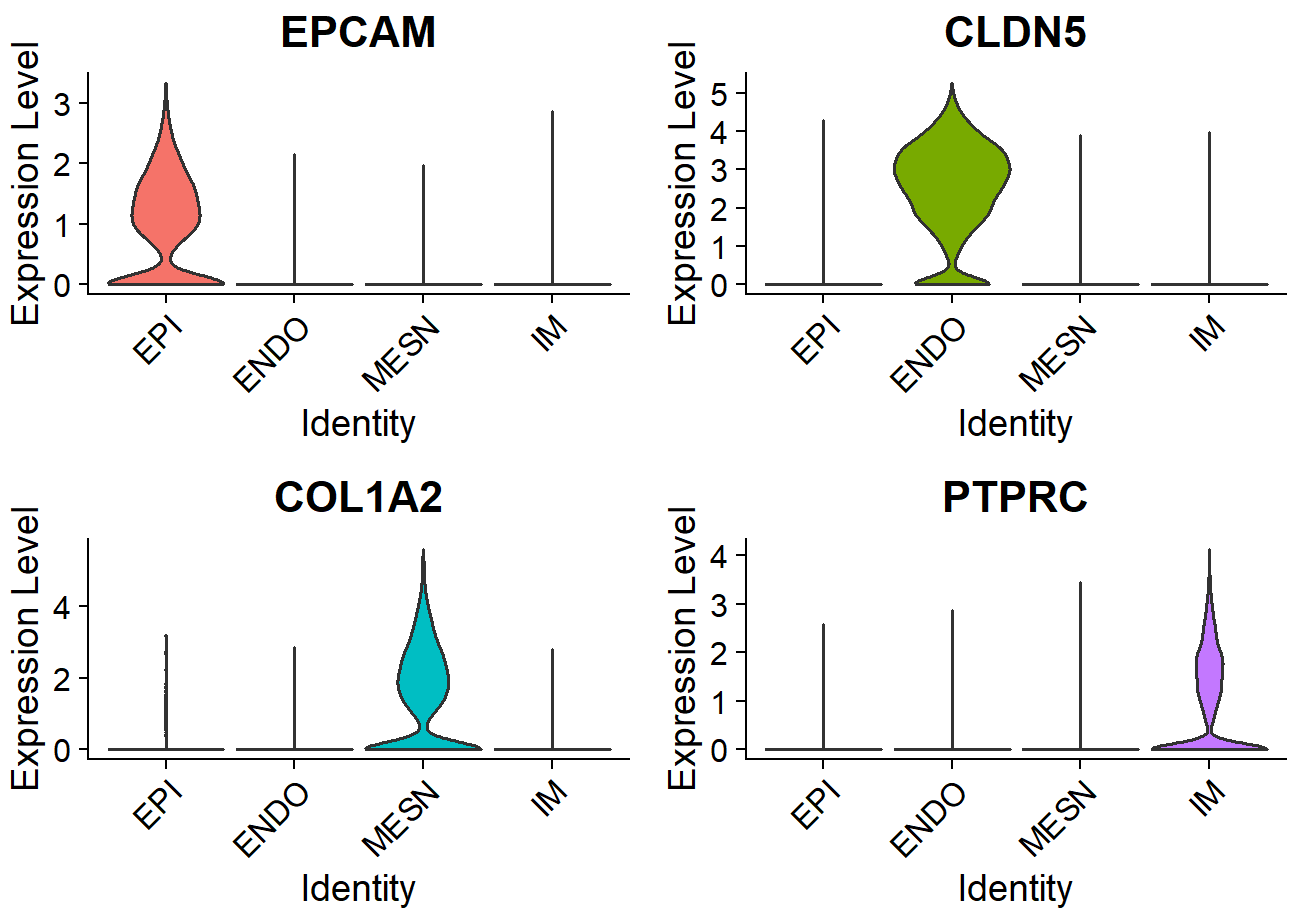
```
FeaturePlot(CONIPF.combined, features = c("DCN", "LUM"), blend = TRUE)
```



Violin Plot

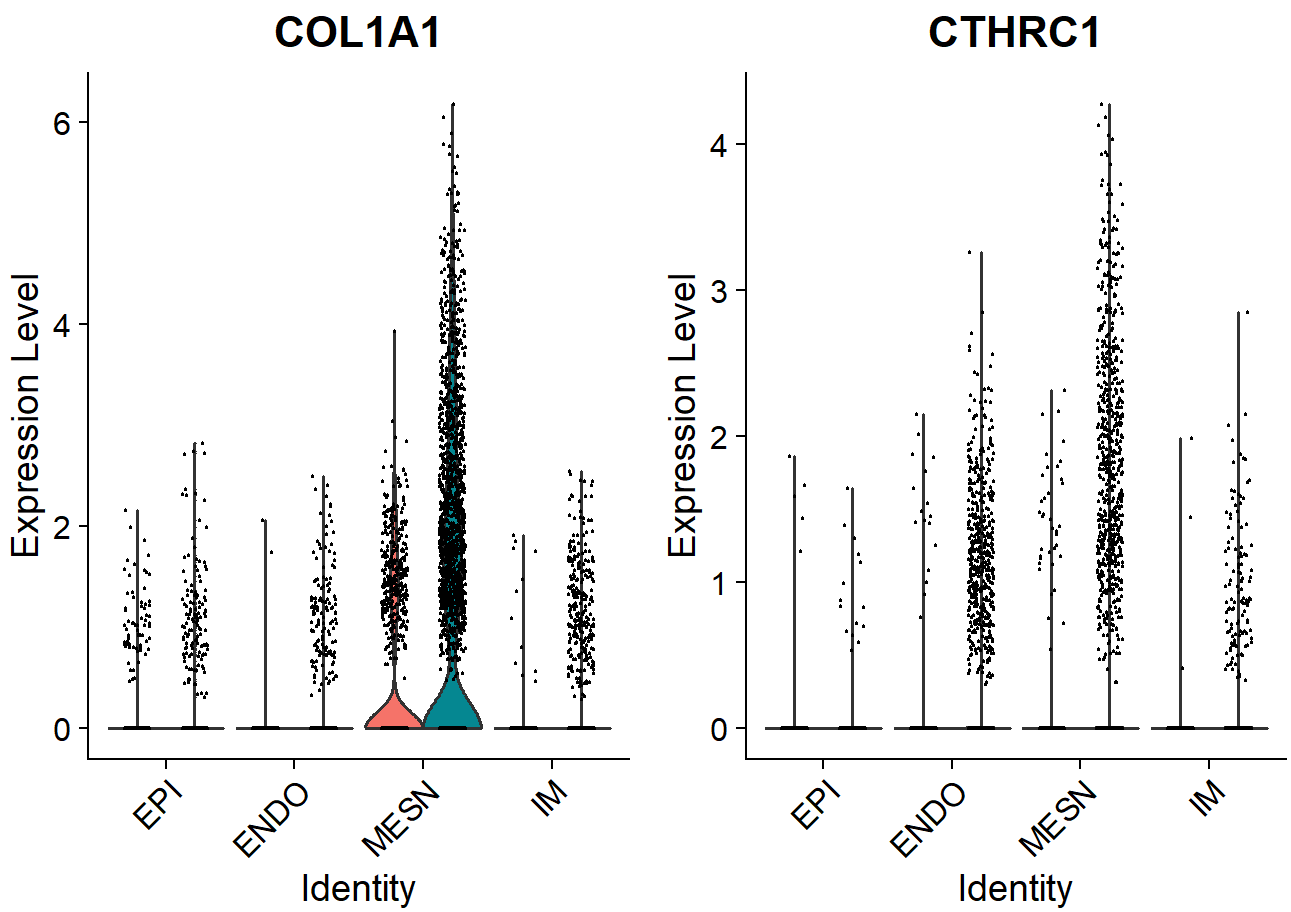
Visualize marker genes are highly expressed in each cell type

```
VlnPlot(CONIPF.combined, features = c("EPCAM", "CLDN5", "COL1A2", "PTPRC"), ncol = 2, pt.size =
0)
```



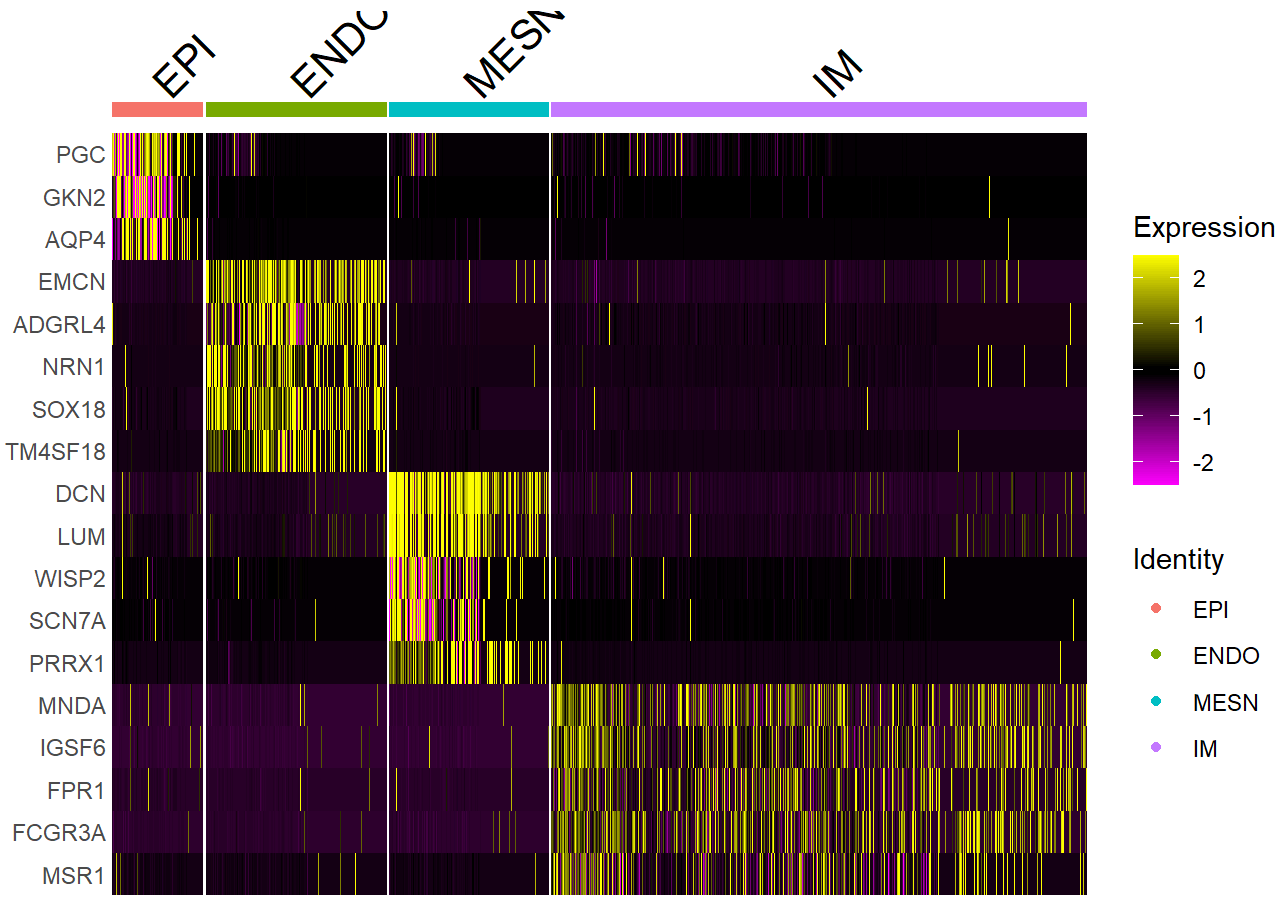
COL1A1 and CTHRC1 gene are upregulated in disease

```
VlnPlot(CONIPF.combined, features = c("COL1A1", "CTHRC1"), split.by = "orig.ident") # left is co
ntrol, right is disease
```



DoHeatmap

```
DefaultAssay(CONIPF.combined) <- "RNA"
CONIPF.combined <- JoinLayers(CONIPF.combined, assay = "RNA") # FindAllMarkers() required JoinLa
yers
CONIPF.combined.markers <- FindAllMarkers(CONIPF.combined, only.pos = TRUE, min.pct = 0.25, logf
c.threshold = 0.25)

# find top 5 genes for each cell type
CONIPF.combined.markers %>%
  group_by(cluster) %>%
  top_n(n = 5, wt = avg_log2FC) -> top5

DefaultAssay(CONIPF.combined) <- "integrated"
DoHeatmap(CONIPF.combined, features = top5$gene)
```



Save Seurat Object as RDS file

It's recommended to save the data as RDS file so that later on we can easily load the RDS file to R. RDS file eliminates the need to save data in the environment which can be memory heavy.

```
# To save as RDS:
saveRDS(CONIPF.combined, file="CONIPF.combined.RDS")
# To read RDS into R:
CONIPF.combined <- readRDS("CONIPF.combined.RDS")
```

Session info

```
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/Chicago
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] future_1.58.0      patchwork_1.3.1    lubridate_1.9.4    forcats_1.0.0
##  [5] stringr_1.5.1      dplyr_1.1.4        purrr_1.0.4        readr_2.1.5
##  [9] tidyr_1.3.1        tibble_3.3.0       ggplot2_4.0.2      tidyverse_2.0.0
## [13] Seurat_5.4.0       SeuratObject_5.3.0 sp_2.2-0
##
## loaded via a namespace (and not attached):
##   [1] RColorBrewer_1.1-3     rstudioapi_0.17.1      jsonlite_2.0.0
##   [4] magrittr_2.0.3         spatstat.utils_3.1-5   ggbeeswarm_0.7.2
##   [7] farver_2.1.2           rmarkdown_2.29         vctrs_0.6.5
##  [10] ROCR_1.0-11            spatstat.explore_3.4-3 htmltools_0.5.8.1
##  [13] sass_0.4.10            sctransform_0.4.2      parallelly_1.45.0
##  [16] KernSmooth_2.23-26     bslib_0.9.0            htmlwidgets_1.6.4
##  [19] ica_1.0-3              plyr_1.8.9             plotly_4.11.0
##  [22] zoo_1.8-14             cachem_1.1.0           igraph_2.1.4
##  [25] mime_0.13              lifecycle_1.0.5        pkgconfig_2.0.3
##  [28] Matrix_1.7-2           R6_2.6.1               fastmap_1.2.0
##  [31] fitdistrplus_1.2-4     shiny_1.11.1           digest_0.6.37
##  [34] tensor_1.5.1           RSpectra_0.16-2        irlba_2.3.5.1
##  [37] labeling_0.4.3         progressr_0.15.1       spatstat.sparse_3.1-0
##  [40] timechange_0.3.0       httr_1.4.7             polyclip_1.10-7
##  [43] abind_1.4-8            compiler_4.4.2         withr_3.0.2
##  [46] S7_0.2.0               fastDummies_1.7.5      R.utils_2.13.0
##  [49] MASS_7.3-65            tools_4.4.2            vipor_0.4.7
##  [52] lmtest_0.9-40          beeswarm_0.4.0         httpuv_1.6.16
##  [55] future.apply_1.20.0    goftest_1.2-3          R.oo_1.27.1
##  [58] glue_1.8.0             nlme_3.1-168           promises_1.3.3
##  [61] grid_4.4.2             Rtsne_0.17             cluster_2.1.8
##  [64] reshape2_1.4.4         generics_0.1.4         gtable_0.3.6
##  [67] spatstat.data_3.1-6    tzdb_0.5.0             R.methodsS3_1.8.2
##  [70] data.table_1.17.6      hms_1.1.3              spatstat.geom_3.5-0
##  [73] RcppAnnoy_0.0.22       ggrepel_0.9.6          RANN_2.6.2
```

```
##  [76] pillar_1.11.1          limma_3.62.2           spam_2.11-1
##  [79] RcppHNSW_0.6.0         later_1.4.2            splines_4.4.2
##  [82] lattice_0.22-6         survival_3.8-3         deldir_2.0-4
##  [85] tidyselect_1.2.1       miniUI_0.1.2           pbapply_1.7-4
##  [88] knitr_1.50             gridExtra_2.3          scattermore_1.2
##  [91] xfun_0.52              statmod_1.5.0          matrixStats_1.5.0
##  [94] stringi_1.8.7          lazyeval_0.2.2         yaml_2.3.10
##  [97] evaluate_1.0.4         codetools_0.2-20       cli_3.6.5
## [100] uwot_0.2.3             xtable_1.8-6           reticulate_1.43.0
## [103] jquerylib_0.1.4        dichromat_2.0-0.1      Rcpp_1.0.14
## [106] globals_0.18.0         spatstat.random_3.4-1  png_0.1-8
## [109] ggrastr_1.0.2          spatstat.univar_3.1-4  parallel_4.4.2
## [112] presto_1.0.0           dotCall64_1.2          listenv_0.9.1
## [115] viridisLite_0.4.3      scales_1.4.0           ggridges_0.5.6
## [118] rlang_1.1.6            cowplot_1.2.0
```